

AL2002 Artificial Intelligence Lab Manual
ver 2.0

Saad Ahmad, BS

Inst., Dept. of Computer Science

National University of Computer & Emerging Sciences

Artificial Intelligence Lab Manual

Saad Ahmad

Instructor, Dept. of Computer Science

National University of Computer & Emerging Sciences

160 Industrial Estate, Peshawar, KPK, Pakistan

Version History

1.0 Spring 2026: Compilation of all previous labs in book format with minor updates & corrections {**Saad Ahmad**}

2.0 Spring 2026: Restructured manual: updated chapter titles, added Simulated Annealing, new CSP chapter (Lab 9), Linear & Logistic Regression in Ch. 13, improved heuristics explanation, added TikZ figures & code snippets {**Saad Ahmad**}

This manual is customized for course AL2002 Artificial Intelligence taught at the National University of Computer & Emerging Sciences, Peshawar, Pakistan. The lab portion of this course provides cover for 1 credit hour. Evaluation for this lab is at discretion of your instructor and/or teacher.

The manual can also be used as a learning tool for other artificial intelligence courses both at undergraduate or graduate level. Each chapter in this manual is intended to serve as a separate lab, although instructors should note that some chapter's require more than one lab sessions to complete.

Contents

I	CONSTRAINT SATISFACTION	5
1	Constraint Satisfaction Problems (CSP)	7
1.1	What is a Constraint Satisfaction Problem?	7
1.2	Formal Definition of a CSP	7
1.3	Variations of Variables, Domains, and Constraints	8
1.3.1	Types of Domains	8
1.3.2	Types of Constraints	8
1.4	Detailed Examples of CSPs	8
1.4.1	Map Coloring Problem	8
1.4.2	N-Queens Problem	9
1.4.3	Cryptarithmic Puzzles	9
1.4.4	Sudoku	9
1.5	Constraint Graphs	9
1.6	Solving CSPs: Inference and Constraint Propagation	10
1.6.1	Node and Arc Consistency	10
1.6.2	The AC-3 Algorithm	10
1.7	Solving CSPs: Backtracking Search	10
1.7.1	Heuristics to Speed Up Backtracking	11
1.8	Backtracking: Implementation Snippet	11

Part I

CONSTRAINT SATISFACTION

Chapter 1

Constraint Satisfaction Problems (CSP)

1.1 What is a Constraint Satisfaction Problem?

A Constraint Satisfaction Problem (CSP) is a mathematical framework for representing and solving problems where we must find states or objects that satisfy a number of constraints or criteria. Unlike many search problems (like route finding) where the state is treated as a "black box" and only evaluated by a goal test, a CSP represents states explicitly. In simple terms, we have decisions to make (variables), options for each decision (domains), and rules dictating which combinations of options are valid (constraints).

Example: Imagine you are tasked with scheduling final exams for a university. The variables are the courses (e.g., CS101, MATH201). The domains are the available time slots and rooms. The constraints are rules like "No student can take two exams at the same time," "A room cannot host two exams simultaneously," and "An exam must be scheduled in a room large enough to hold all enrolled students." Your job is to assign a valid time and room to each course so that all constraints are satisfied.

By modeling a problem as a CSP, we can leverage general-purpose, powerful heuristics and constraint propagation algorithms rather than writing a custom, problem-specific search algorithm from scratch.

1.2 Formal Definition of a CSP

Formally, a Constraint Satisfaction Problem is defined as a tuple $\langle X, D, C \rangle$, where:

- **X** is a set of variables, $X = \{X_1, X_2, \dots, X_n\}$.
- **D** is a set of domains, $D = \{D_1, D_2, \dots, D_n\}$, such that each variable X_i has a corresponding domain D_i . A domain consists of a set of allowable values $\{v_1, v_2, \dots, v_k\}$ for the variable.
- **C** is a set of constraints that specify allowable combinations of values. Each constraint $C_i \in C$ consists of a pair $\langle \text{scope}, \text{rel} \rangle$, where *scope* is a tuple of variables that participate in the constraint, and *rel* is a relation that defines the values that those variables can simultaneously take.

An **assignment** assigns values to some or all variables.

- A **consistent** (or legal) assignment is one that does not violate any constraints.

- A **complete** assignment gives a value to every variable.
- A **solution** to a CSP is a consistent and complete assignment.

1.3 Variations of Variables, Domains, and Constraints

1.3.1 Types of Domains

Domains can vary significantly depending on the nature of the problem:

- **Discrete and Finite Domains:** Variables can only take a limited number of specific, separate values. Map coloring and Sudoku use discrete, finite domains. These are the most common in basic CSPs.
- **Discrete and Infinite Domains:** Variables can take infinitely many distinct values, such as integers. An example is job scheduling where the domain is the set of integer time steps.
- **Continuous Domains:** Variables can take real numbers. Common in operations research and linear programming, where constraints are often represented as inequalities (e.g., $X_1 + X_2 \leq 5$).

1.3.2 Types of Constraints

Constraints are categorized by how many variables they involve (their arity):

- **Unary Constraints:** Restrict the value of a single variable. Example: "Region WA cannot be green" ($WA \neq \text{Green}$). Unary constraints are typically handled by simply removing the violating values from the variable's initial domain.
- **Binary Constraints:** Relate two variables. Example: "WA and NT must have different colors" ($WA \neq NT$).
- **Higher-order (Global) Constraints:** Involve three or more variables. Example: In a Cryptarithmic puzzle, or the `AllDiff` constraint requiring that all variables in a set (e.g., all numbers in a Sudoku row) take distinct values.

1.4 Detailed Examples of CSPs

1.4.1 Map Coloring Problem

Color a map so that no two adjacent regions share the same color.

- **Variables:** $X = \{WA, NT, SA, Q, NSW, V, T\}$ (regions of Australia)
- **Domain:** $D_i = \{\text{Red}, \text{Green}, \text{Blue}\}$ for all variables
- **Constraints:** Adjacent regions must have different colors. Examples: $WA \neq NT$, $WA \neq SA$, $NT \neq SA$, etc.

1.4.2 N-Queens Problem

Place N queens on an $N \times N$ chessboard so that no two queens attack each other.

- **Variables:** $X = \{Q_1, Q_2, \dots, Q_N\}$, where variable Q_i represents the row position of the queen in the i -th column.
- **Domain:** $D_i = \{1, 2, \dots, N\}$ for each queen.
- **Constraints:** No two queens can share a row ($Q_i \neq Q_j$) or a diagonal ($|Q_i - Q_j| \neq |i - j|$).

1.4.3 Cryptarithmic Puzzles

A puzzle where letters are mapped to distinct digits such that an arithmetic equation holds. A classic example is: TWO + TWO = FOUR.

- **Variables:** $X = \{T, W, O, F, U, R, C_1, C_2, C_3\}$, where letters represent digits and C_i represent carry-over limits.
- **Domain:** $D_{\text{letters}} = \{0, 1, \dots, 9\}$ (with $T \neq 0, F \neq 0$). $D_{\text{carries}} = \{0, 1\}$.
- **Constraints:** AllDiff(T, W, O, F, U, R), $O + O = R + 10 \cdot C_1$, $C_1 + W + W = U + 10 \cdot C_2$, and so forth.

1.4.4 Sudoku

Fill a 9×9 grid so that each row, column, and 3×3 sub-grid contains the digits $1 \dots 9$ exactly once.

- **Variables:** $X = \{X_{11}, X_{12}, \dots, X_{99}\}$ for each cell.
- **Domain:** $D_{ij} = \{1, 2, \dots, 9\}$. Many variables are already pre-assigned values in the initial state.
- **Constraints:** AllDiff for each row, AllDiff for each column, and AllDiff for each 3×3 block.

1.5 Constraint Graphs

When a CSP only involves binary constraints, we can represent it visually using a **constraint graph**.

- **Nodes** represent the variables.
- **Edges** represent the constraints between any two variables.

This representation is highly advantageous because it allows us to analyze the structure of the problem. If a problem's constraint graph is disconnected into completely separate subgraphs, they can be solved independently. If the graph is a tree, it can be solved in linear time.

For map coloring, the constraint graph has one node per region, and edges connect neighboring regions. The following diagram shows a simple constraint graph:

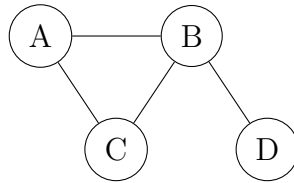


Figure 1.1: A simple constraint graph for four regions. A, B, and C are mutually adjacent, while D is only adjacent to B.

For constraints involving more than two variables, a standard graph is insufficient, and we use a **hypergraph** where hyperedges can connect three or more nodes simultaneously.

1.6 Solving CSPs: Inference and Constraint Propagation

Before or during search, we can use the constraints to reduce the domains of the variables through **constraint propagation** (also known as inference). This process eliminates values from domains that can never be part of a consistent solution.

1.6.1 Node and Arc Consistency

These are techniques used to "clean up" the domains of variables by filtering out invalid options early, making the search algorithm run drastically faster.

- **Node Consistency (Unary Constraints):** A variable is node-consistent if all values in its domain satisfy its unary constraints.

Example: If the domain of region WA is {Red, Green, Blue} and there is a unary constraint "WA $\neq$ Green", making it node-consistent simply means removing "Green" from its domain. The valid domain becomes {Red, Blue}.

- **Arc Consistency (Binary Constraints):** A variable X_i is arc-consistent with respect to another variable X_j if, for *every* value left in the domain of X_i , there exists *at least one* valid corresponding value in the domain of X_j that satisfies the binary constraint between them.

Example: Suppose $A \neq B$. If the domain of A is {Red, Green} and the domain of B is {Red}, is A arc-consistent with B? If we pick Green for A, B can be Red. That works. But if we pick Red for A, there is no valid color left for B! Because choosing Red for A forces a dead-end for B, we must delete Red from A's domain. A's arc-consistent domain becomes just {Green}.

1.6.2 The AC-3 Algorithm

The most famous constraint propagation algorithm is **AC-3** (Arc Consistency Algorithm #3). It maintains a queue of all pairs of constrained variables (arcs). It pops an arc (X_i, X_j) and makes X_i arc-consistent with X_j . If it removes any value from D_i , it must re-add all arcs pointing to X_i back into the queue (because pruning D_i might suddenly make a neighbor variable inconsistent). This drastically reduces the search space before or during backtracking search.

1.7 Solving CSPs: Backtracking Search

Backtracking search is the foundational algorithm for solving CSPs. It operates as a depth-first search that chooses values for one variable at a time, but with a critical optimization: it immediately stops (backtracks) and abandons an assignment path if a constraint is violated.

The steps are:

1. **Select** an unassigned variable.
2. **Iterate** through its domain. Assign a value.
3. **Check constraints.** If the assignment violates no constraints, recursively continue to the next variable.
4. **Backtrack.** If a violation occurs, or the recursive call returns failure, remove the assignment and try the next value in the domain.
5. If all variables are assigned successfully, return the solution.

1.7.1 Heuristics to Speed Up Backtracking

Standard backtracking can be slow for large problems. We use intelligent heuristics to answer two key questions: *Which variable should we pick next?* and *Which value should we try first?*

- **Minimum Remaining Values (MRV) Heuristic:** Pick the variable with the fewest valid options left in its domain. This "fail-first" approach helps identify dead ends early, pruning the search tree immensely.
- **Degree Heuristic:** If MRV gives a tie, pick the variable that is involved in the largest number of constraints with other unassigned variables. This helps reduce the branching factor of future choices.
- **Least Constraining Value (LCV) Heuristic:** Once a variable is picked, order its values by choosing the one that rules out the fewest options for neighboring unassigned variables. This "fail-last" approach increases the chance of finding a solution quickly rather than exploring doomed subtrees.

1.8 Backtracking: Implementation Snippet

The following Python-like pseudocode incorporates checking heuristics during the backtracking process:

```

1 def backtrack(assignment, csp):
2     # If all variables are assigned, we found a solution
3     if len(assignment) == len(csp.variables):
4         return assignment
5
6     # Heuristics: MRV and Degree Heuristics
7     var = select_unassigned_variable_MRV_Degree(assignment, csp)
8
9     # Heuristics: Least Constraining Value
10    for value in order_domain_values_LCV(var, assignment, csp):
11        if is_consistent(var, value, assignment, csp):
12            assignment[var] = value

```

```
13
14     # Optional: apply inference like AC-3 here (Forward Checking)
15     inferences = apply_constraint_propagation(var, value, csp)
16
17     if inferences != FAILURE:
18         csp.add_inferences(inferences)
19
20         result = backtrack(assignment, csp)
21         if result is not None:
22             return result
23
24         csp.remove_inferences(inferences)
25
26     del assignment[var] # Undo assignment
27
28     return None # No solution found on this branch
```

Listing 1.1: Backtracking with Heuristics for CSP

Key Additions:

- `select_unassigned_variable_MRV_Degree`: Chooses variables intelligently to fail early and save time.
- `order_domain_values_LCV`: Chooses values that leave the most options open for neighbors.
- `apply_constraint_propagation`: Performing inference, like checking arc consistency, dynamically as assignments are made (an approach called Maintaining Arc Consistency or MAC).

Bibliography

- [1] Agents in artificial intelligence.
- [2] Artificial intelligence.
- [3] Artificial intelligence.
- [4] Artificial intelligence - agents and environments.
- [5] Cs6-1 lab 06.
- [6] Networkx - python software package for complex networks.
- [7] Types of artificial intelligence.
- [8] What is intelligent agent in ai - types and function.